



Ejercicio 1 (4 puntos)

Considere una función en ASM de 32 bits denominada CheckLong. A esta función se le pasan dos parámetros por la pila:

- El primer parámetro es la dirección de inicio de un vector de caracteres.
- El segundo parámetro es un valor entero que indica la cantidad de elementos del vector de caracteres.

La función CheckLong debe recorrer el vector hasta encontrar un cero, luego compara con el valor siguiente al cero (el cual debería ser la cantidad de elementos del vector) y verifica si la cuenta es correcta comparando este número con la cuenta realizada.

Checklong **retorna** la diferencia que hubo entre el valor calculado y el valor informado. Es decir, si son iguales retorna cero, si el valor calculado es mayor al informado retorna la diferencia en valor positivo, si el valor informado es mayor, retorna la diferencia en valor negativo.

CALCULADO - INFORMADO.

Además se le pide a Ud. que está rutina tiene que quedar lista para poder ser llamada desde un programa en C

Se pide:

- Indique de qué manera recibirá CheckLong cada uno de los parámetros a la función.
- Programar utilizando lenguaje ASM el código de la función CheckLong usando el siguiente fragmento de programa:

```
section .data
msg: db "Hola Mundo", 0
len: db 10
```

no entendí al final si le tengo que pasar los parámetros desde C o sólo uso eso.

- Escriba un programa en C en un archivo llamado *principal.c* que solo llame a la función realizada en el punto b) y que imprima en pantalla si el chequeo de la longitud fue exitoso, o no.
- ¿Que consideraciones debe tener para poder compilar y linkeditar ambos archivos? ¿El archivo .asm y el archivo .c?

```
b) ii) section .data
      msg: db "Hola Mundo", 0
      len: db 10
section .text
global  checkLong
```

```
checkLong:
      mov     eax, 0,
```

```
loop:
      mov     ebx, [msg + eax];
      cmp     ebx, 0;
      je      fin;
      inc     eax;
```

```
fin:
      mov     ebx, [msg + eax + 1];
      sub     eax, ebx;
      ret;
```

```
d) nasm -f elf32 checklong.asm
gcc -C -m32 principal.c
gcc -m32 checklong.o principal.o -o programa
```

checklong no debe tener la etiqueta `_start`.

- a) Primero se pushea al stack el segundo parámetro (cant. elementos) y luego se pushea el primer parámetro (dir. del inicio del vector). Es decir: en $EBP + 12$ = cant elementos
 $EBP + 8$ = dir. inicio vector

Así sería si la llamo desde C:

```
b) section .text
global  checkLong
```

checkLong:

```
      push   ebp;
      mov     ebp, esp; }
```

Armado de SF

```
      push   ebx;           → Preservo ebx
```

```
      push   edx;
```

```
      mov     ebx, [ebp + 8]; → En ebx guardo el 1° elem del array
```

```
      mov     edx, [ebp + 12]; → valor informado.
```

```
      mov     eax, 0;       → contador
```

loop:

```
      cmp     [ebx], '\0',
```

```
      je      fin;
```

```
      inc     eax; → incremento el cont. caracteres
```

```
      inc     ebx; → paso al sig. elem del array.
```

```
      jmp     loop;
```

fin:

```
      sub     eax, edx; → guardo el valor de retorno en EAX
```

```
      pop     edx;
```

```
      pop     ebx;
```

```
      mov     esp, ebp; }
```

```
      pop     ebp;
```

```
      ret;
```

```
section .data
```

```
msg: db "Hola Mundo", 0
```

```
len: db 10
```

```
c) principal .c
```

```
#include <stdio.h>
```

```
extern int checkLong(char[] msg, int long);
```

```
int main(void){
    char[] msg = "Hola Mundo";
    int len = 10;
```

```
    if (checkLong(msg, len) == 0){
```

```
        printf("El chequeo fue exitoso\n");
```

```
    }else{
```

```
        printf("El chequeo no fue exitoso\n");
```

```
    }
```

```
    return 0;
```

```
}
```

Ejercicio 2 (3 puntos)

Dado el siguiente programa que se compila para un procesador de 32 bits:

```
#include<stdio.h>

int calculo(int param1,int param2, char tipo) {
    EBP+8  EBP+12  EBP+16
    int resul;

    if(tipo=='s')
        resul=param1+param2;
```

```
    else
        resul=param1*param2;

    return resul;
}

int main(void) {

    int valor1=7;
    int valor2=3;
    char operacion;
    printf("Ingrese el tipo de operación: 's' suma y 'm' multiplicar : \n");
    scanf("%c",&operacion);
    printf("Resultado: %d\n",calculo(valor1, valor2, operacion));
    return 0;
}
```

a. Explique con dibujos respectivos el contenido de la pila, durante toda la ejecución del programa.
b. Muestre los registros del microprocesador que intervienen en los dibujos de la pila.

¿debería agregarle el canary?

b) intervienen los registros EBP, EBP y EAX. (y EBP por ahí)

28 RET		
27 POP EBP		
26 MOV ESP,EBPm	RET main	23 call printf
25 MOV EAX, 0	"Resultado: %d \n"	22 Push "string"
24 ADD ESP, 4 * 2	EAX	21 Push EAX (resul. de cálculo)
	RET main	19 call calculo
	[EBP - 4]	18 Push valor1
20 ADD ESP, 4 * 3	[EBP - 8]	13 Push valor2
	[EBP - 12]	16 Push operación
	RET main	14 call scanf
15 ADD ESP, 8 (borra parámetros)	"%c"	13 Push "%c"
	[EBP - 12]	12 Push [EBP - 12] (*operación)
	RET main	10 call printf
11 ADD ESP, 4 (para borrar el parámetro)	"ingrese el tipo de op..."	9
	operación = EAX 15	8 SUB ESP, 4
	valor2 = 3 7	6 SUB ESP, 4
	valor1 = 7 5	4 SUB ESP, 4
push EBPso 2	EBPso	← EBPmain MOV EBPm, ESP 2
	RETso	1

Un alumno que todavía no cursó esta materia le pide a ud. que le explique:

a) ¿Qué es una system call? ¿Quién la escribió? ¿Dónde se encuentran ubicadas?

b) ¿Cómo se llama a una system call desde Assembler de Linux? Muestre con un ejemplo de llamada a read.

c) ¿Qué es el número de ID que aparece en la clase teórica?

Como ud no recuerda de memoria el funcionamiento ingresa a un sistema operativo Linux y ejecuta los comandos "man 2 read" y obtiene la siguiente salida:

a) Una syscall es una interfaz entre un programa de usuario y el sistema operativo. Sirve para solicitar servicios al kernel del SO, como accesos al hardware, operaciones de entrada y salida, etc.

Las escribieron los desarrolladores del SO en cuestión

Están ubicadas en el kernel del SO.

b) Desde Assembler de Linux, para hacer una llamada a una syscall hay que cargar en EAX el número de syscall, en otros registros (de ser necesario) los parámetros de la syscall según corresponda y por último, ejecutar la línea "int \$0h".

Ejemplo : si queremos hacer read():

```
MOV EAX, 3;          syscall número 3 para read
MOV EBX, 0;          file descriptor (0= stdin)
MOV ECX, buffer;
MOV EDX, buffer.size;
```

int \$0h

```
section .data
buffer      db 32
buffer.size equ 32
```


Ejercicio 1 (4 puntos)

Se dispone de un microprocesador de tipo Intel de 8 bits de bus de datos, 32 bits de bus de direcciones. El cual ya tiene conectado un módulo de ROM de 256MB x 8 se encuentra al final del mapa de memoria. Este módulo ya se encuentra decodificado. Ud. no tiene que hacer nada con este módulo de memoria.

Por una cuestión de crecimiento del proyecto nos dicen que se necesita agregar:

- a) Un módulo de RAM de 1G x 8
- b) Un módulo de RAM de 256M x 8
- c) Un módulo de ROM de 256M x 8

Por una cuestión de espacio y economía Ud. debe usar **la mínima cantidad de decodificadores** posibles para que el proyecto sea un éxito. Solo puede usar **decodificadores**.

Se pide en este orden que:

- 1) Dibuje el mapa de memoria con el módulo de ROM pre-existente.
- 2) Dibuje un diagrama esquemático de los integrados utilizados para la RAM y ROM nuevas (líneas de datos, direcciones, etc) Debe colocar nombres a **todas** las líneas de circuito utilizadas.
- 3) Resuelva la decodificación con estas condiciones y dibuje el circuito obtenido.

Intel 32 bits x 8bits

ROM 256MB x 8

$$32 \text{ bits} \Rightarrow 2^{32} = 2^{30} \cdot 2^2 = 4G$$

$$2^8 \cdot 2^{20} = 2^{28}$$

dir min: 0000 0000h

dir min: 0000 0000h

dir max: FFFF FFFFh

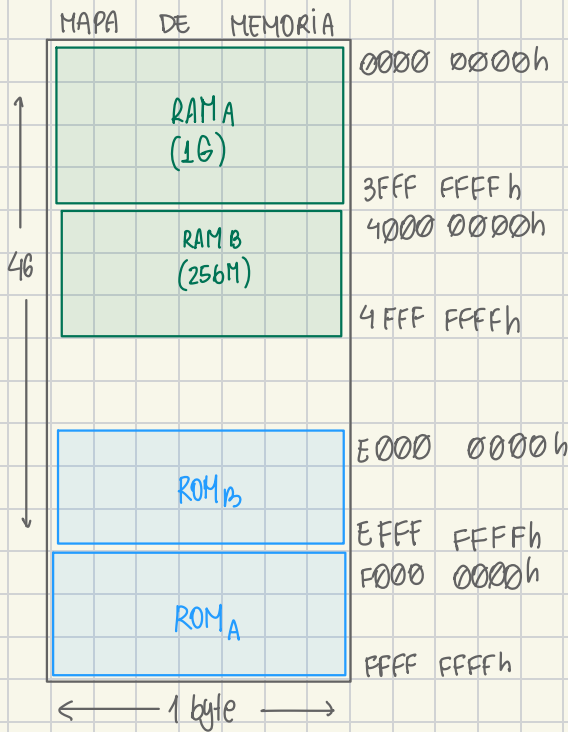
dir max: 0FFF FFFFh

RAM 1G x 8: 2^{30}

min: 0000 0000h

max: 3FFF FFFFh

+ 3FFF FFFF
+ 0FFF FFFF



ROM A:

1111
1111

0 0 0 0 0 0 0 0
F F F F F F F F

A31-28=1 →
IOM=0

No hay que decodificarlo

ROM B:

1110
1110

0 0 0 0 0 0 0 0
F F F F F F F F

A31-29=1
A28=0
IOM=0

Voy a usar dos decos de 3 a 8

RAM A:

0000
0011

0 0 0 0 0 0 0 0
F F F F F F F F

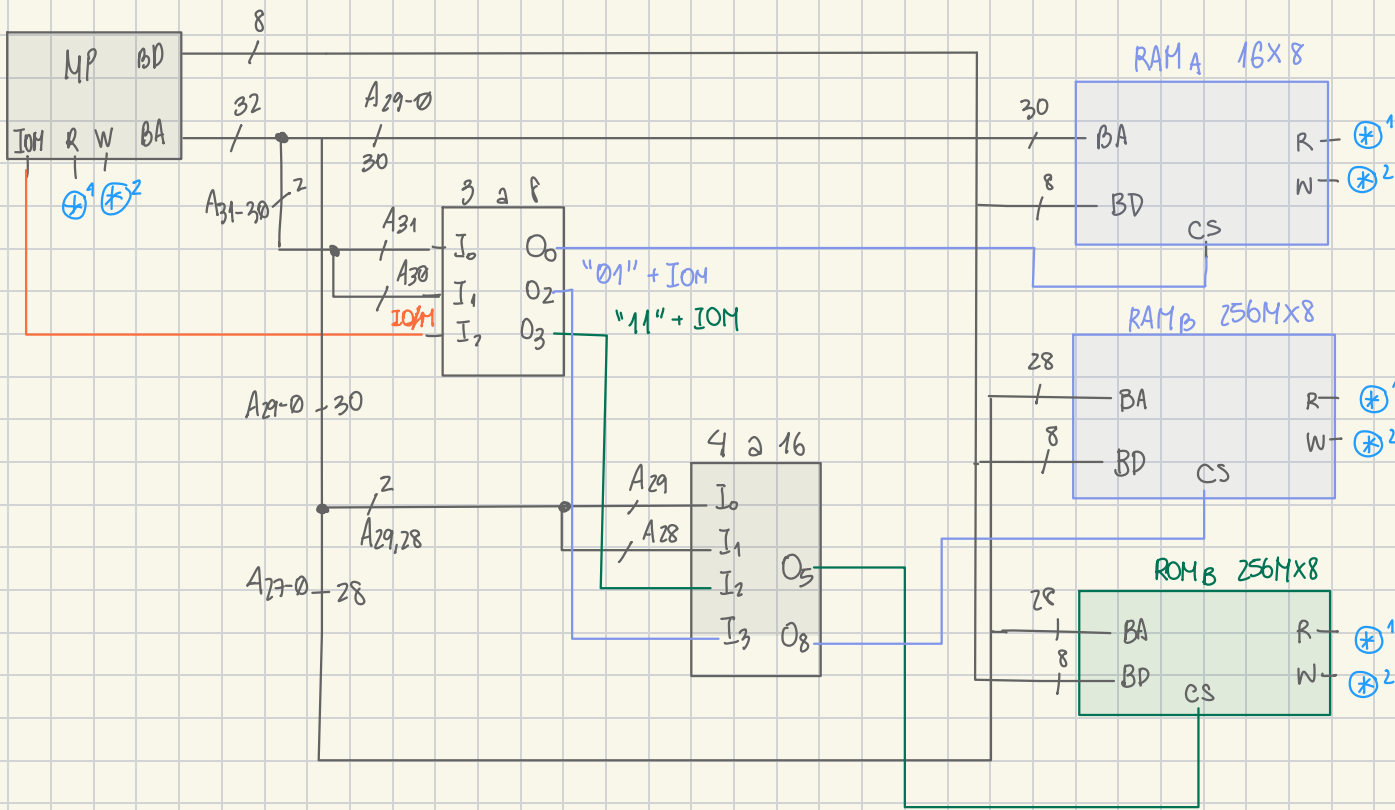
A31-30=0
IOM=0

RAM B:

0100
0100

0 0 0 0 0 0 0 0
F F F F F F F F

A30=1
A31,29,28=0
IOM=0



Ejercicio 1

Ud. ya dispone de un microprocesador tipo Intel de 8 bits de bus de datos y 16 bits de bus de direcciones. **Un programador necesita correr un programa cuando inicia el procesador.** El fabricante del procesador informa en su hoja de datos que al iniciarse el micro el registro IP toma el valor FFF0h. Conversando con el programador le dice a Ud. que va a necesitar 4KBytes de ROM y 8KBytes de RAM

Se pide en este orden que:

- Explique que le tendría que avisar al programador en cuanto a la primera instrucción que debe correr su programa, debido a la condición de IP=FFF0h al inicio.
- Dibuje un diagrama individual de los integrados que utilizará antes de realizar la decodificación y el mapa de memoria.
- Resuelva la decodificación completa (sin imágenes) con estas condiciones y dibuje el circuito obtenido. Solo puede **usar como máximo dos decodificadores** para resolver el ejercicio.

a) Le tengo que avisar al programador que la primera instrucción de su programa debe estar en la dir. FFF0h.

b) MAPA DE MEMORIA

RAM
8KB

0000h
1FFFh

ROM
4KB

F000h
FFFFh

4KB ROM: $4K = 2^2 \cdot 2^{10} = 2^{12}$
dir min: 0000h
dir máx: 0FFFh
8KB RAM: $8K = 2^3 \cdot 2^{10} = 2^{13}$
dir min: 0000h
dir máx: 1FFFh

ROM:

11 11	000
11 11	FFF

$A_{15-12}=1$
 $I/O/M=0$

RAM:

0000	000
0001	FFF

$A_{15-13}=0$
 $I/O/M=0$

1^{11}
FFF 0
+ 0FF F
10FEF → Me pasé

ROM

FFFF

- 0FFF

F000

↓ deco de 2 a 4

↓ deco de 4 a 16

2P-2021-1Q

Pregunta 1

Ud. ya dispone de un microprocesador tipo Intel de 8 bits de bus de datos y 32 bits de bus de direcciones.

A este procesador se deberá conectarle los siguientes dispositivos:

- 256 MB de memoria ROM teniendo en cuenta que dispone de chips de $64\text{M} \times 8$
- 2 GB de memoria RAM teniendo en cuenta que dispone de chips de $256\text{M} \times 4$
- 1 GB de memoria de video a partir de la dirección 0x00000000h
- 4 puertos de E/S de 8 bits para acceso al teclado (la ubicación queda a elección del alumno)

Nos informan que al iniciarse el microprocesador el registro EIP toma el valor FFFF0000h.

Se pide en este orden que:

- Dibuje el mapa de memoria final.
- Resuelva la decodificación completa (sin imágenes) de todos los dispositivos usando decodificadores **solamente de hasta 4 entradas y en la menor cantidad posible.**
- Desarrolle un programa Assembler ejemplo que lea los 4 puertos de E/S del teclado y los coloque en la memoria de video

RAMA: 0100 → 1+ → 0 ⇒ 1
B: 0101 → 8 ⇒ 9
C: 0110 → 4 ⇒ 5
D: 0111 → 12 ⇒ 13

E: 1000 → 2+ → 0 ⇒ 2
F: 1001 → 8 ⇒ 10
G: 1010 → 4 ⇒ 6
H: 1011 → 12 ⇒ 14

$2^{32} = 2^{30} \cdot 2^2 = 4G$

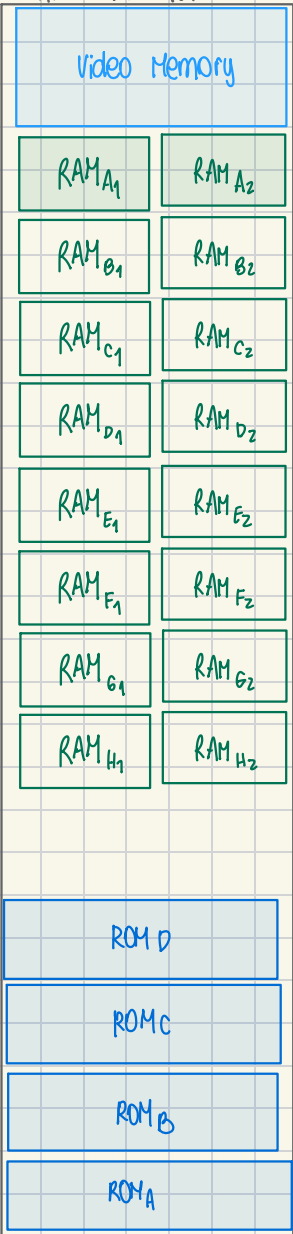
ROM: $64M = 2^6 \cdot 2^{20} = 2^{26}$ → Necesito 4
min = 0000 0000h
máx = 03 FFFF FFh

RAM: $256M = 2^8 \cdot 2^{20} = 2^{28}$ → $2G = 2 \cdot 2^{30} = 2^{31}$
min = 0000 0000h
máx = 0FFF FFFFh
⇒ $2^{28} \cdot 2^3 = 2^{31}$
8 ⇒ como son de 256Mx4 y necesito 8 bits de BD, uso el doble ⇒ 16

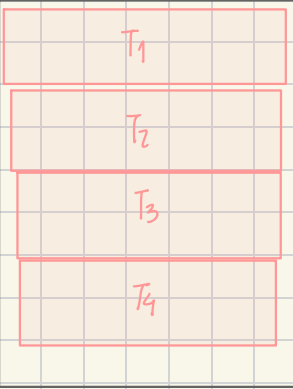
V/Mem: 1G = 2^{30}
min = 0000 0000h
máx = 3FFF FFFFh

Puerto E/S: min = 0000 0000h
máx = 0000 00FFh

MAPA DE MEMORIA



MAPA E/S



0000 0000h	0000 0000h
0000 00FF h	0000 00FF h
0000 0100 h	0000 01FF h
0000 0200 h	0000 02FF h
0000 0300 h	0000 03FF h

T ₁ : 0000 0000h	0000 00FFh	A ₃₁₋₈ = 0	I/O/M = 1
T ₂ : 0000 0001h	0000 000Fh	A ₃₁₋₉ = 0	A ₈ = 1
T ₃ : 0000 0010h	0000 001Fh	A _{31-10,8} = 0	A ₉ = 1
T ₄ : 0000 0011h	0000 001Fh	A ₃₁₋₁₀ = 0	A _{9,8} = 1

los que varían son A₈ y A₉

Video Mem:	0000 0000	0000 0000	A _{31,30} = 0	I/O/M = 0
	0011	FFF FFFF		

RAM A:	0100 0000 0000	A _{31,29,28} = 0
	0100 FFF FFFF	A ₃₀ = 0
		I/O/M = 0

RAM B:	0101 0000 0000	A _{31,29} = 0
	0101 FFF ...	A _{30,28} = 1
		I/O/M = 0

RAM C:	0110 0000 0000	A _{31,28} = 0
	0110 FF ...	A _{30,29} = 1
		I/O/M = 0

RAM D:	0111 0000 0000	A ₃₁ = 0
	0111 FF ...	A ₃₀₋₂₈ = 1
		I/O/M = 0

RAM E:	1000 0000 0000	A ₃₁ = 1
	1000 FFF FFFF	A ₃₀₋₂₈ = 0
		I/O/M = 0

RAM F:	1001 0000 0000	A _{31,28} = 1
	1001 FFF ...	A _{30,29} = 0
		I/O/M = 0

RAM G:	1010 0000 0000	A _{31,29} = 1
	1010 FF ...	A _{30,28} = 0
		I/O/M = 0

RAM H:	1011 0000 0000	A _{31,29,28} = 1
	1011 FF ...	A ₃₀ = 0
		I/O/M = 0

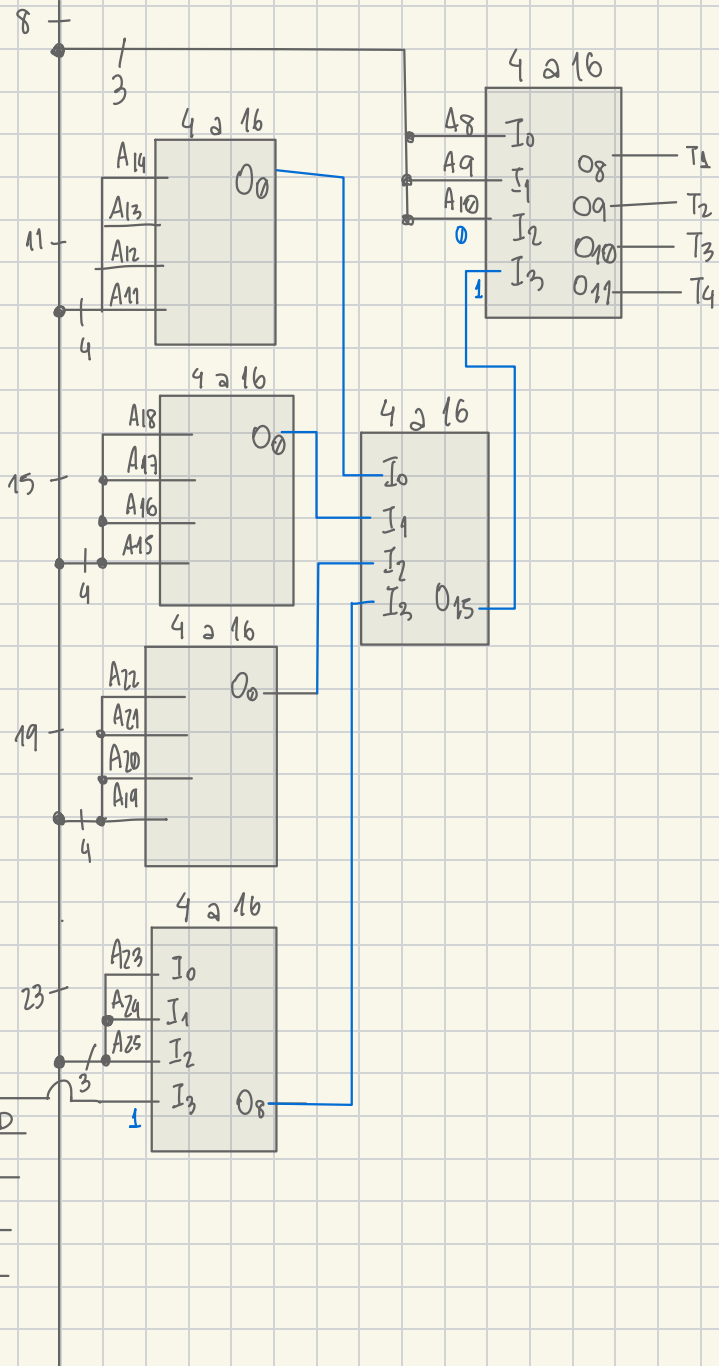
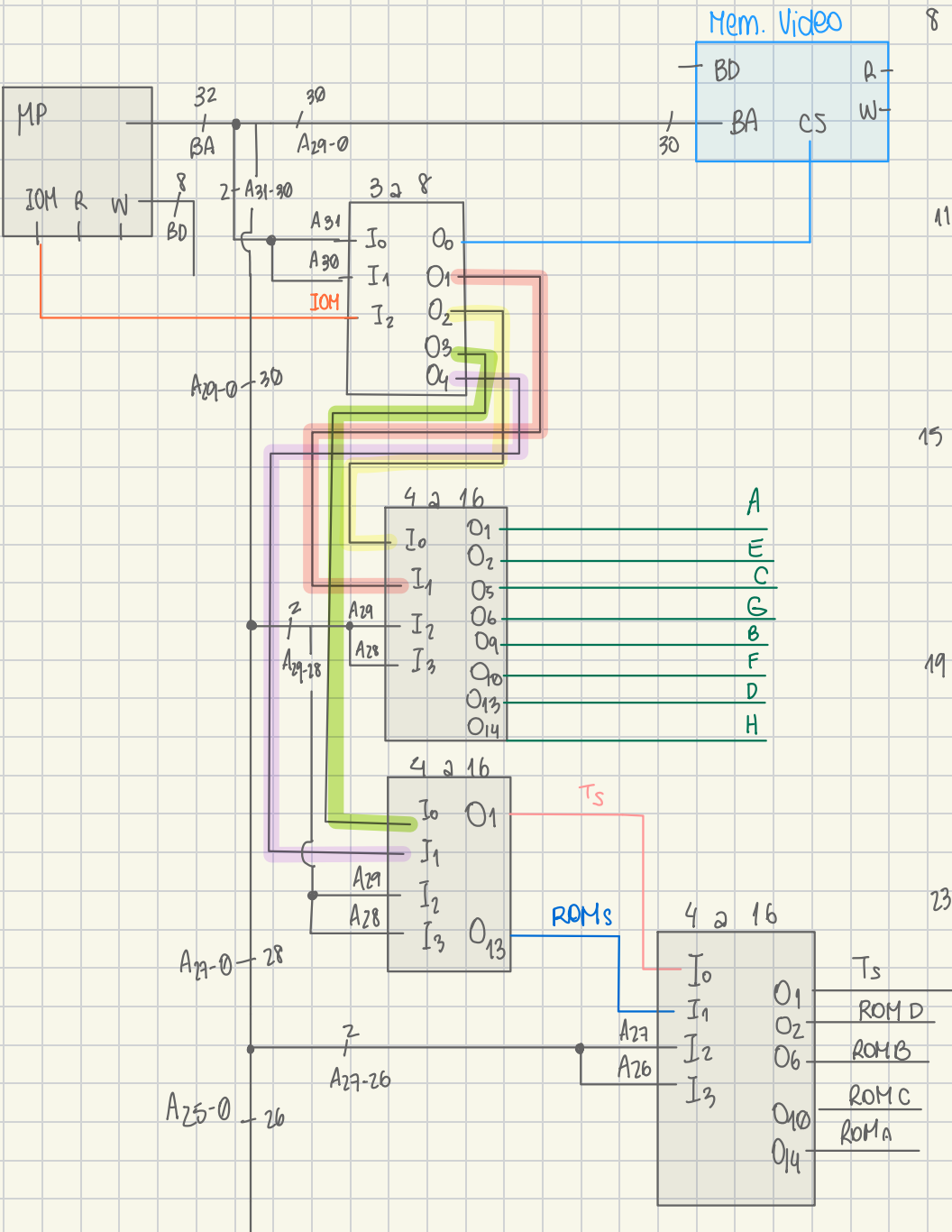
ROM A:	1111 1100 00 0000	A ₃₁₋₂₆ = 1
	1111 1111 FF FFFF	I/O/M = 0

ROM B:	1111 1000 0000 0000	A ₃₁₋₂₇ = 1
	1111 1011 FFFF FFFF	A ₂₆ = 0
		I/O/M = 0

ROM C:	1111 0100 00 0000	A _{31-28, A₂₆} = 1
	1111 0111 FF FFFF	A ₂₇ = 0
		I/O/M = 0

ROM D:	1111 0000 00 0000	A ₃₁₋₂₈ = 1
	1111 0011 FF FFFF	A ₂₇₋₂₆ = 0
		I/O/M = 0

3 2 8 ③ } cubren los 1° 4 bits y el IOM
4 2 16 ④ → demás bits



32 bits $\Rightarrow 2^{32} = 2^2 \cdot 2^{30} = 46 \text{ totales} = 4 \cdot 2^{10} \cdot 2^{10} \cdot 2^{10} = 2^{22} \cdot 2^{10}$
 ROM: 256K = $2^8 \cdot 2^{10} = 2^{18}$

- $$46-256K = 4194304K - 256K = 4194048K = 4095,75MB = 3,996G$$

$$4194048K - 2^{21}K - 2^{20}K = 1048320K - 2^{19}K = 524032K$$

- $$26B = 2^1 \cdot 2^{30} = 2^{21} K$$
- $$16B = 2^{20} \cdot 2^{10} = 2^{20} K$$

ROM: min: 0000 0000 h
max: 0003 FFFF h

EBP	
RET	+4
ARGC	+8
ARGV	+12

Ejercicio 3 (3 puntos) ARQ 32

Dado el siguiente programa compilado con GCC sin ningún parámetro adicional más que el archivo fuente: → Tiene Canary

```
#include<stdio.h>
```

```
int resta (int a) {
```

```
    int operando=9;
```

```
    return operando - a;
```

```
int main() {
```

```
    int a = resta(resta(9));
```

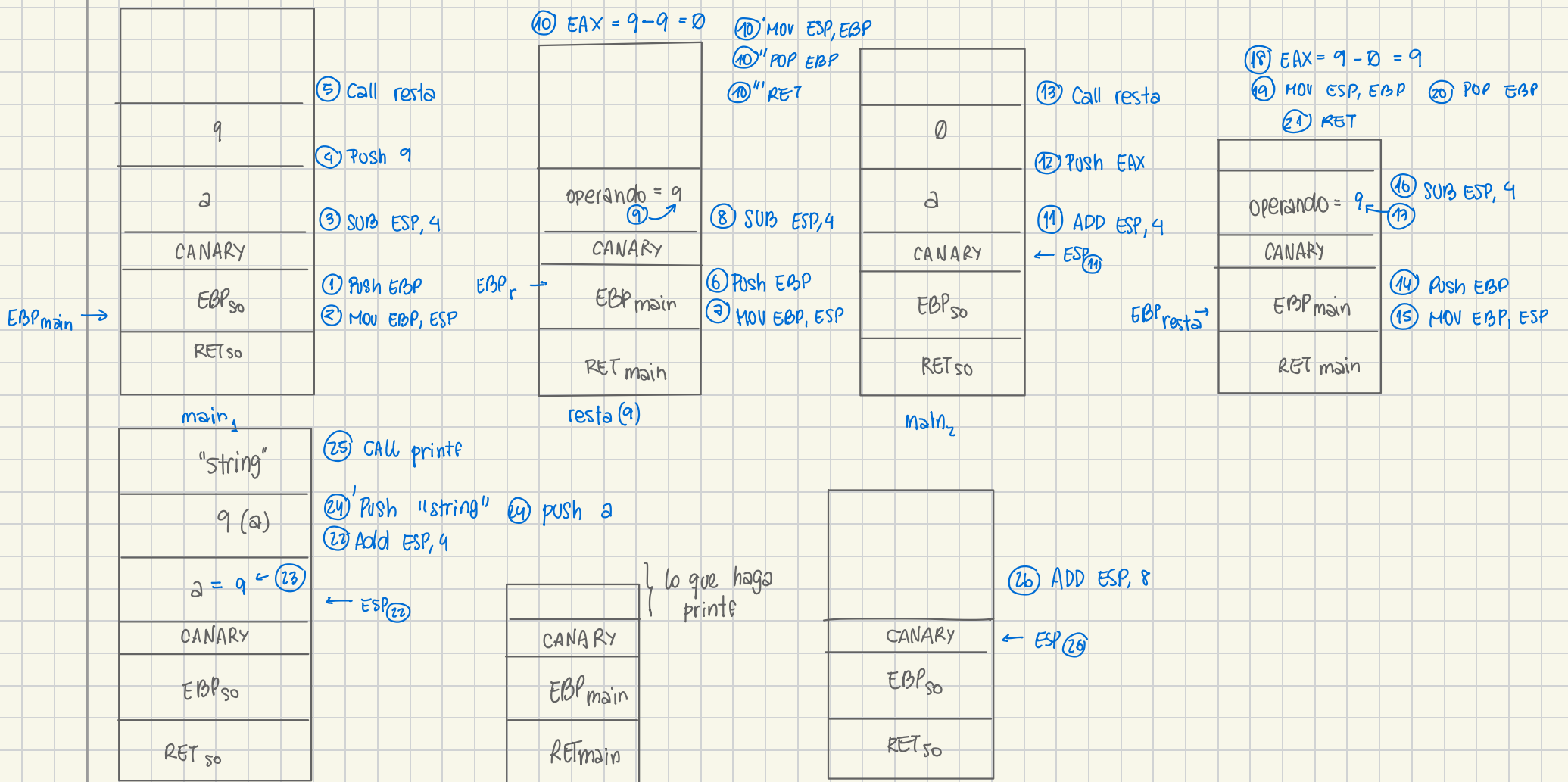
```
    printf("Resultado: %d\n", a);
```

```
    return ; }
```

a) Explique con dibujos respectivos el contenido de la pila, durante toda la ejecución del programa. Muestre los valores de los registros del microprocesador que intervienen en los dibujos de la pila.

b) Muestre los valores que retorna cada función y la forma en que lo hace.

c) Muestre la salida en pantalla del programa.



c) "Resultado: 9"

Ejercicio 1 (4 puntos)

Se le pide realizar un **programa en assembler** (Intel 32 bits) que haga lo siguiente:

- Leer la **humedad ambiente** cada 1 segundo. Use una **system call** para generar el tiempo de espera.
- Ajuste el valor leído de humedad según: $H_{correcta} = H_{leida} / 7$
- Avisar en pantalla si la humedad leída en cada momento **coincide con algún valor en el arreglo** definido en el siguiente fragmento de código

```
sección .data
arreglo      dd    10, 20, 30, 40, 50
cant_arreglo dd    ($-arreglo)/4
```

(donde estoy parado - arreglo) / 4

Otra información que se le brinda

ID	syscall	ebx → tiempo solicitado	ecx → puede ser null
162	sys_nanosleep	const struct timespec *req	struct timespec * _Nullable rem

ebp+4

ebp+8

struct timespec	Cantidad de segundos 4 bytes	Cantidad de nanosegundos 4 bytes
-----------------	---------------------------------	-------------------------------------

Cómo leer la humedad. Linux le permite a usted manejar la entrada analógica donde se lee la humedad ambiente, leyendo un archivo. A continuación se detalla cual es el archivo que debe utilizar:

```
/sys/bus/iio/in_voltage0_raw
```

Usted **no requiere programar la parte de manejo del archivo**: ya se le entrega una función escrita usando convenciones de C para pasaje de argumentos y retorno del valor:

```
int get_humedad(int resolución, char * filename);
// retorna valor de la humedad
// resolución: usar siempre el valor 16 (resolución = 16 bits)
// filename: "/sys/bus/iio/in_voltage0_raw"
```

Las únicas funciones que puede usar son:

- get_humedad
- print: funciona exactamente igual que la función de librería estándar de C para intel 32.
- system calls: no hay restricciones

Si requiere alguna otra función, debe implementarla usted.

→ Hecho en Visual :D

2020-1R

Ejercicio 1

→ Tiene IO/M

Se dispone de un microprocesador de tipo Intel de 8 bits de bus de datos y 32 bits de bus de direcciones. El dueño del proyecto nos brinda 4 (cuatro) módulos de memoria RAM, cada uno de ellos de 256M x 8 y nos pide que los ubiquemos a partir de la dirección 0h en forma contigua para lograr 1GByte de RAM. Además tenemos que decodificar un módulo de ROM de 1G x 8 para el código que corre al encender el sistema, por una cuestión de diseño el módulo de ROM debe comenzar en la mitad del mapa de memoria.

Además le pide que ubique en la zona del **mapa de memoria** que ud. desee un periférico que necesita 64 MB de direcciones.

Se pide en este orden que:

- Dibuje un diagrama individual de los integrados que utilizará antes de realizar la decodificación y el mapa de memoria.
- Resuelva la decodificación completa (sin imágenes) con estas condiciones y dibuje el circuito obtenido.
- Se solicita que utiliza la mínima cantidad de decodificadores para la solución sino será rechazada por el líder de proyecto.

MP: 32 bits = 4G direcciones. = 2^{32} dir

RAM: $256\text{M} = 2^8 \cdot 2^{20} = 2^{28}$ dir.

min = 00000000h

$$\text{máx} = 0\text{FFF FFFFh}$$

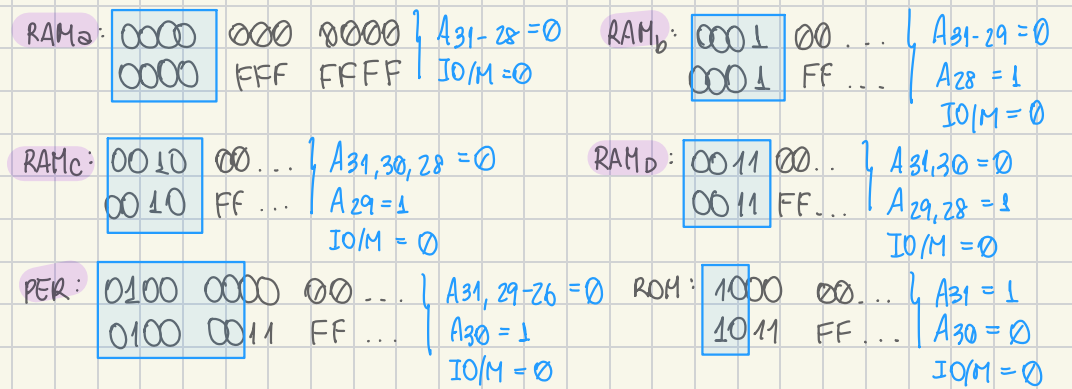
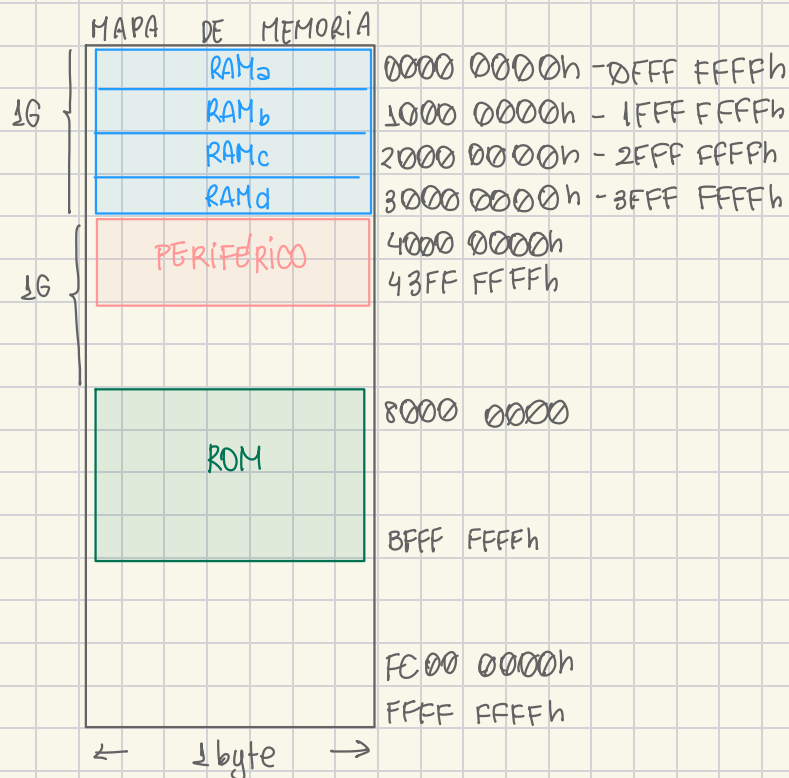
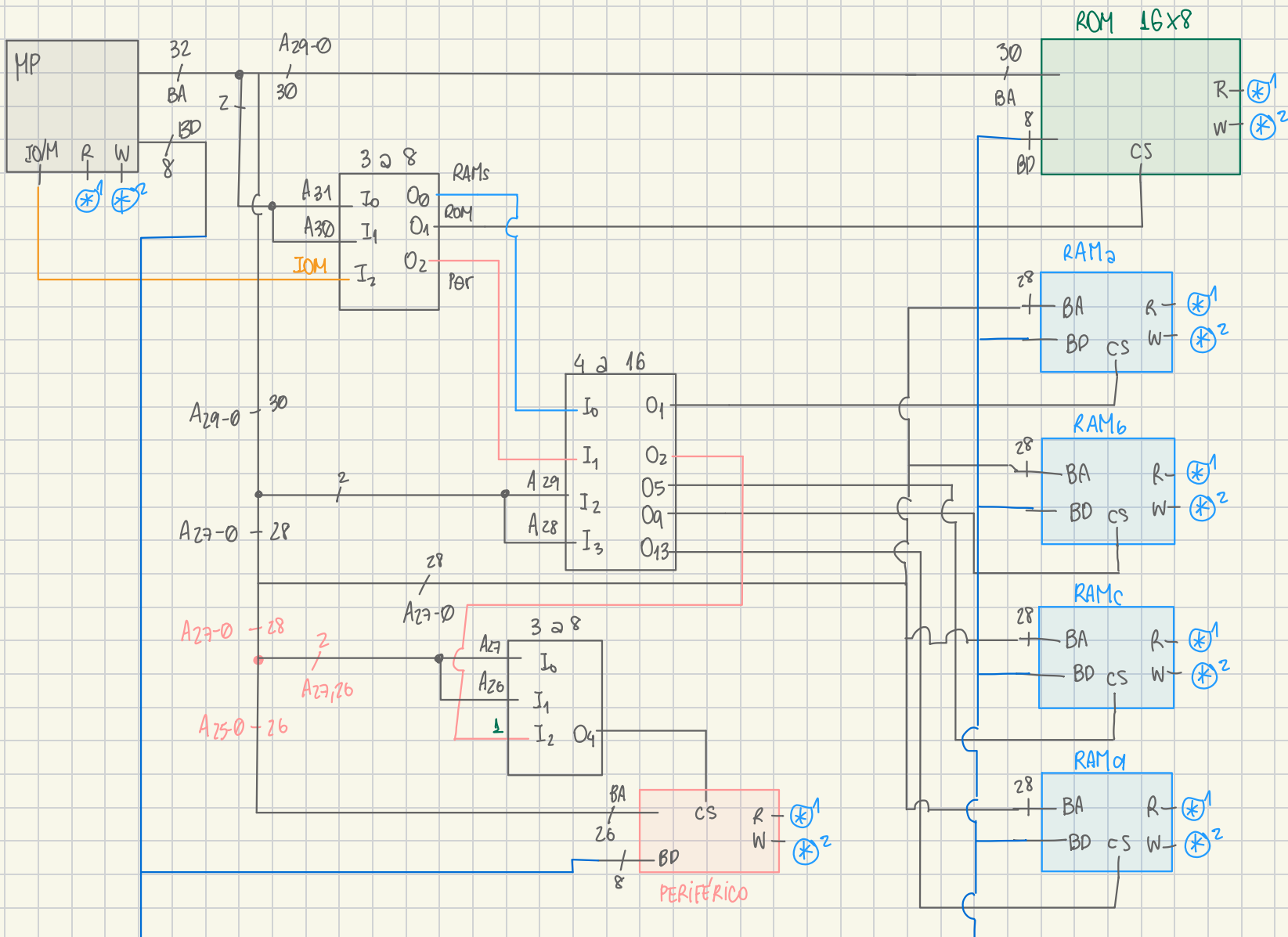
ROM: $16 = 2^{30}$ dir

$$min = 0000 \ 0000h$$
$$m\Delta x = 3FF \quad FF FFh$$

PER: $64M = 2^6 \cdot 2^{20} = 2^{26}$

min = 0000 0000h

max = 03FF FFFFh


$$32\text{ f} \rightarrow A_{31,30} + \text{IOM}$$


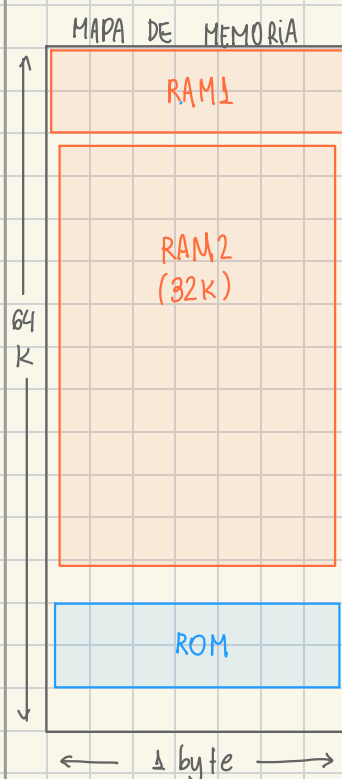
Ud. ya dispone de un microprocesador tipo Intel de 8 bits de bus de datos y 16 bits de bus de direcciones. Le entregan un sistema que ya tiene 8 KB de memoria RAM colocados a partir de la posición 0000h del mapa de memoria y ya tiene hecho el circuito decodificador para esa RAM, que llamaremos RAM1. El sistema no posee memoria ROM.

Le informan que al iniciarse el microprocesador el registro IP toma el valor F000h y le piden que al iniciar el micro se corra un programa que tendrá como máximo 2 KB de ROM, además le dicen que no hay presupuesto y que todo los insumos deben ser ajustados a su mínimo indispensable.

Por otro lado le piden que agregue al sistema que ya tiene 8 KB de RAM1, otro modulo de RAM (llamado RAM2), solo uno, pero que sea contiguo a la RAM1. Se le pide que este módulo de RAM2 ocupe la máxima cantidad de espacio posible en el mapa de memoria, pero siempre cumpliendo la condición de que solo puede usar decodificadores para activar su CS (Chip Select).

Se pide en este orden que:

- Dibuje el mapa de memoria final.
- Explique el porqué del tamaño del módulo RAM2 agregado.
- Resuelva la decodificación completa (sin imágenes) con estas condiciones y dibuje el circuito obtenido utilizando solamente decodificadores de 3x8. No se puede usar la decodificación existente para RAM1.



RAM1: $8K = 2^3 \cdot 2^{10} = 2^{13}$ dir
 min = 0000h
 max = 1FFFh

ROM: $2K = 2^1 \cdot 2^{10} = 2^{11}$ dir
 min = 0000h
 max = 07FFh

RAM2: F000h
 - 1FFFh
 0001h dir $\Rightarrow 53249d = 26 \cdot 624 \times 2 + 1$

$\Rightarrow$ min: 1111 0000 00 } $A_{15}-A_2 = 1$
 max: 1111 0111 FF } $A_{11} = 0$
 $IO/M = 0$

$2^{15} = 32768 \rightarrow$ lo máximo que puedo agregar = 32K
 $2^{16} = 65536$

• 2^{15} dir
 min = 0000h
 max = 7FFFh

• Como tiene que ser contigua a la RAM1, debe arrancar en la pos 2000h
 min = 0010 0000
 max = 1001 FFFF

$1^0 \rightarrow 0$ $2^0 \rightarrow 0$ $3^0 \rightarrow 1 \Rightarrow 4^0 = 0$
 $1 \rightarrow 1$ $0 \Rightarrow 4 = 1$

deco de 3 a 8 $\Rightarrow$ divida el mapa de memoria en 8 sectores ($2^3 = 8$)

Ud. ya dispone de un microprocesador tipo Intel de 8 bits de bus de datos y 32 bits de bus de direcciones.

A este procesador se deberá conectarle los siguientes dispositivos:

- 256 MB de memoria ROM teniendo en cuenta de dispone de chips de 256 Mb x 8 → uso una sola
- 1GB de memoria de video a partir de la dirección 0x80000000h → mitad del mapa.
- un dispositivo de acceso a la red al cual se accede a través de 16 puertos de E/S → IO/M
- Se deberá colocar la mayor cantidad de memoria RAM posible usando

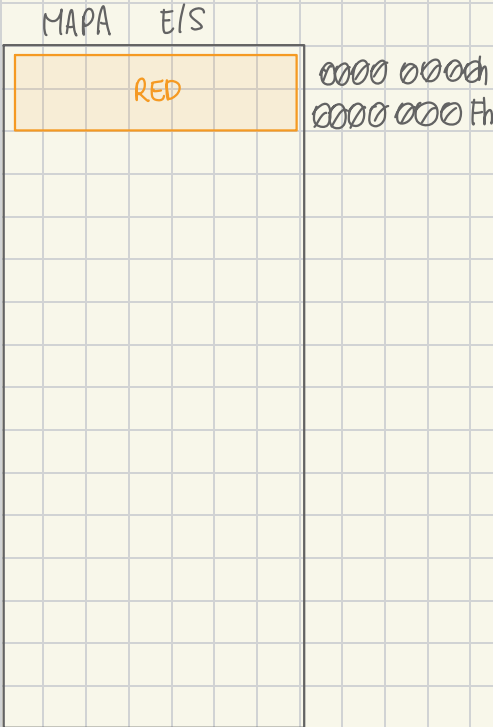
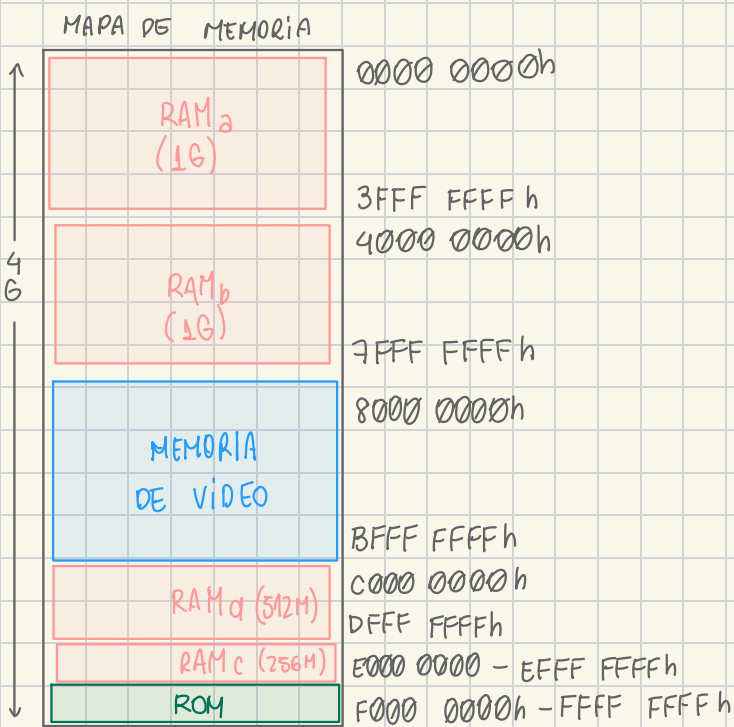
los siguientes chips disponibles:

- 64 Mb x 8
- 128 Mb x 8
- 256 Mb x 8 → ①
- 512 Mb x 8 → ①
- 1 Gb x 8 → ②

Nos informan que al iniciarse el microprocesador el registro EIP toma el valor FFFF0000h.

Se pide en este orden que:

- Dibuje el mapa de memoria final.
- Resuelva la decodificación completa (sin imágenes) de todos los dispositivos usando decodificadores solamente de hasta 4 entradas y en la menor cantidad posible.



Video: $1G = 2^{30}$ dir
min: 0000 0000
máx: 3FFF FFFF

ROM: $2^8 \cdot 2^{20} = 2^{28}$ dir
min: 0000 0000
máx: 0FFF FFFF

RAMd: $2^9 \cdot 2^{20} = 2^{29}$ dir
máx: 1FFF FFFF

RED: 16 puertos = 2^4 puertos
máx: 0000 000F

RAMa: $\begin{matrix} 0000 & 00 \dots \\ 0011 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31-30} = 0 \\ IOM = 0 \end{matrix} \right.$

RAMb: $\begin{matrix} 0100 & 00 \dots \\ 0111 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31} = 0 \\ A_{30} = 1 \\ IOM = 0 \end{matrix} \right.$

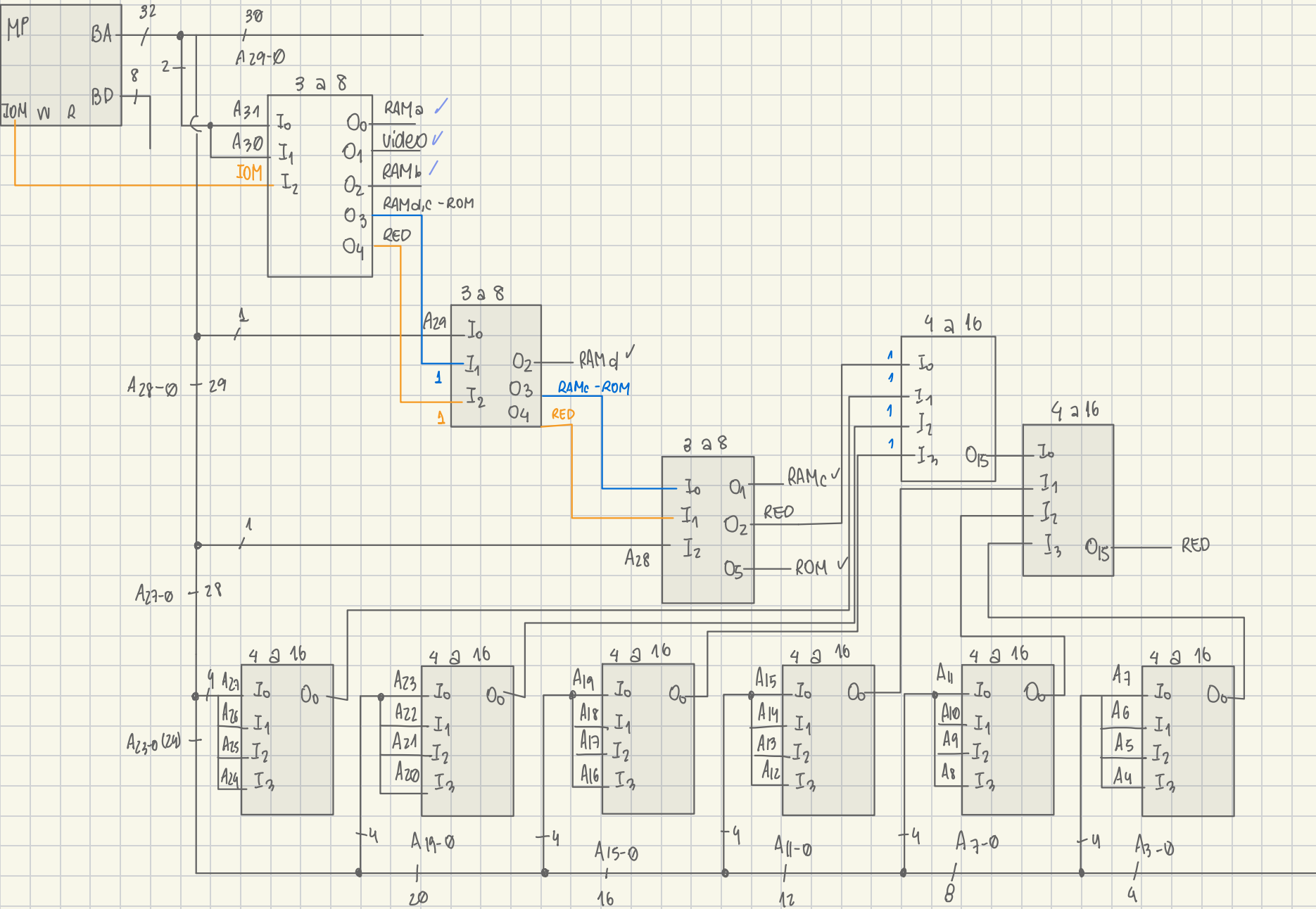
Video: $\begin{matrix} 1000 & 00 \dots \\ 1011 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31} = 1 \\ A_{30} = 0 \\ IOM = 0 \end{matrix} \right.$

RAMd: $\begin{matrix} 1100 & 00 \dots \\ 1101 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31,30} = 1 \\ A_{29} = 0 \\ IOM = 0 \end{matrix} \right.$

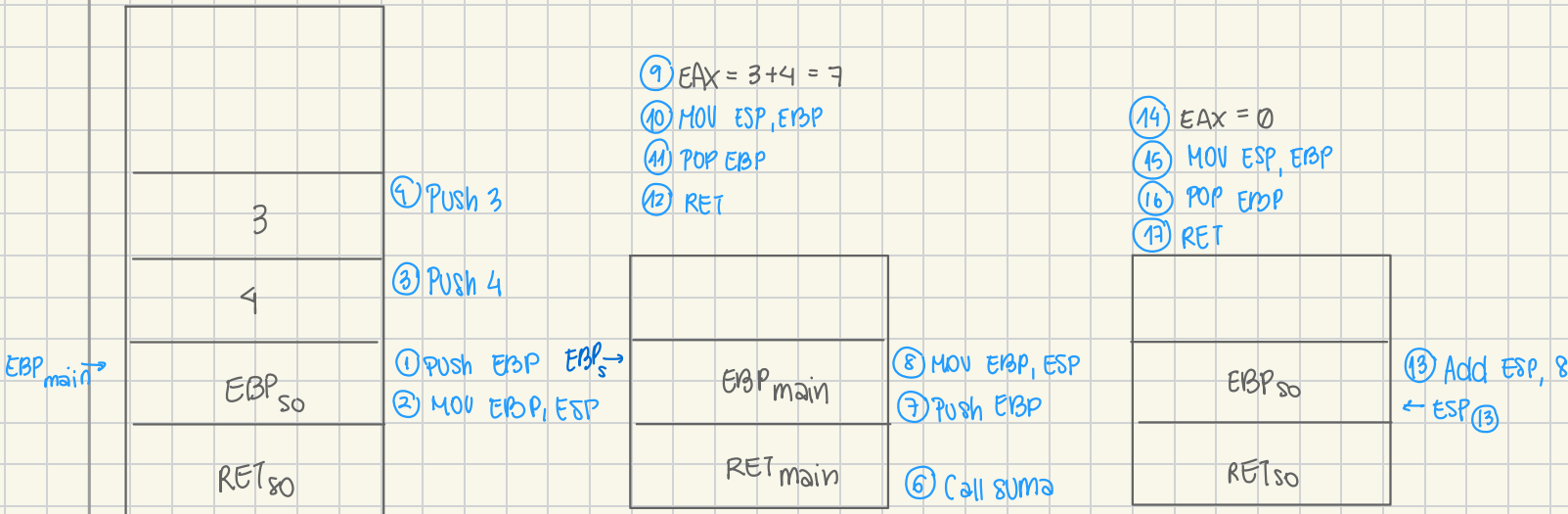
RAMc: $\begin{matrix} 1110 & 00 \dots \\ 1110 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31-29} = 1 \\ A_{28} = 0 \\ IOM = 0 \end{matrix} \right.$

ROM: $\begin{matrix} 1111 & 00 \dots \\ 1111 & FF \dots \end{matrix} \left\{ \begin{matrix} A_{31-28} = 1 \\ IOM = 0 \end{matrix} \right.$

RED: $\begin{matrix} 0000 & 0000 \\ 0000 & 000F \end{matrix} \left\{ \begin{matrix} A_{31-4} = 0 \\ IOM = 1 \end{matrix} \right.$




```
//detalle1.c
// Programa para analizar en detalle los stack frame
int suma( int sum1, int sum2)
{
    return sum1+sum2;
}
int main(void)
{
    suma(3,4);
    return 0;
}
```



1R-2022

Pregunta 1

Dado el siguiente programa que se compila en C para un procesador Intel de 32 bits:

```
#include<stdio.h>

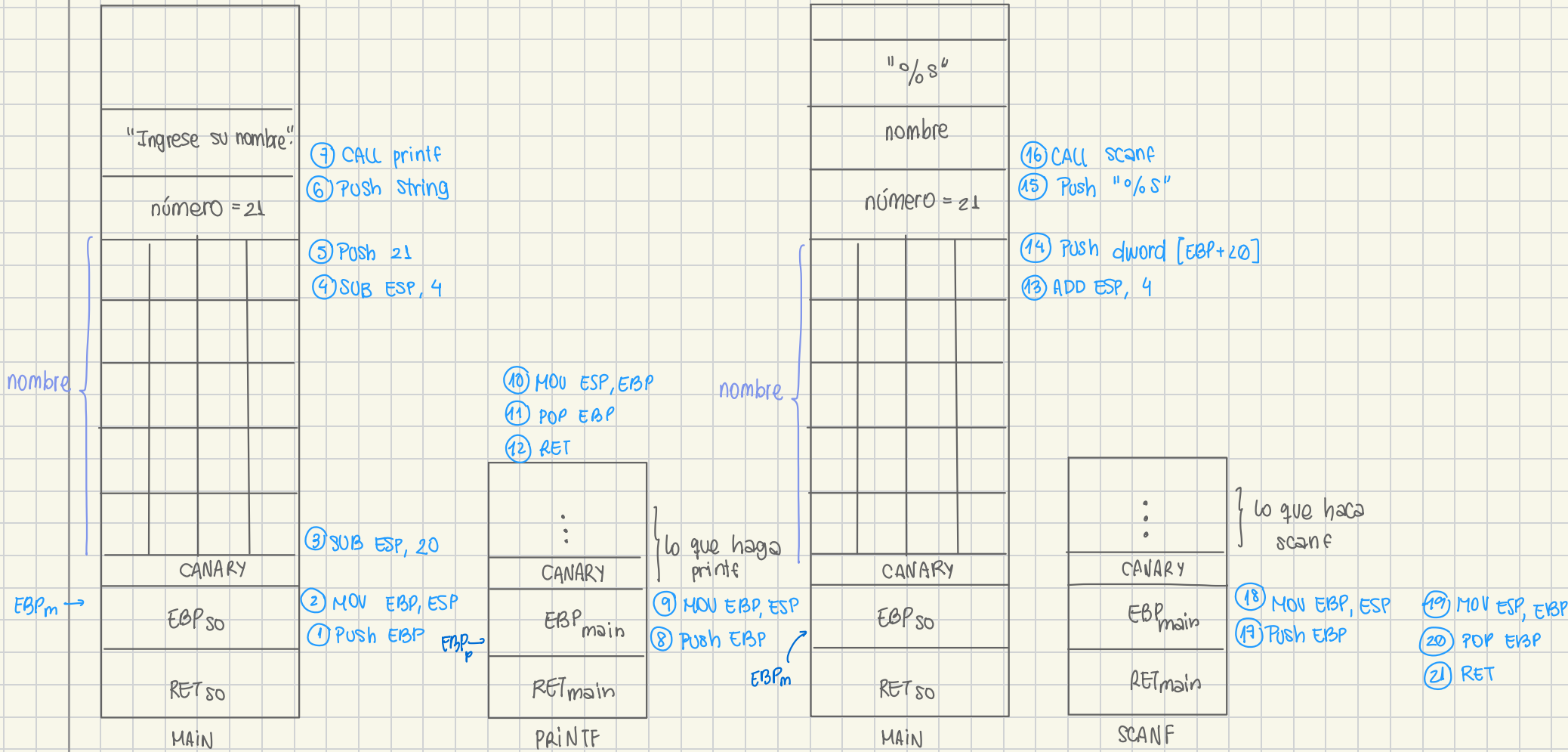
int paso2(int var2)
{
    return var2*2;
}

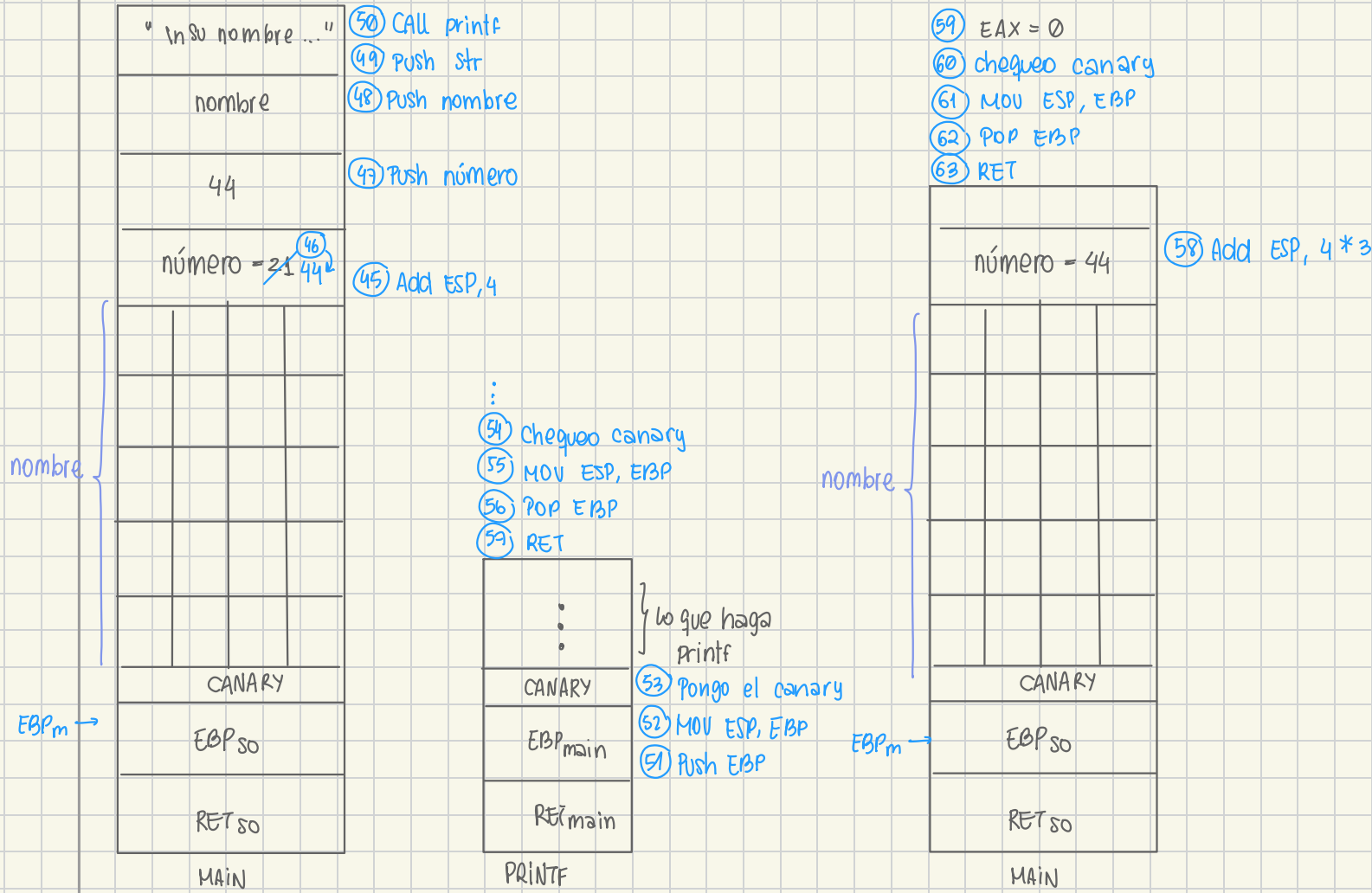
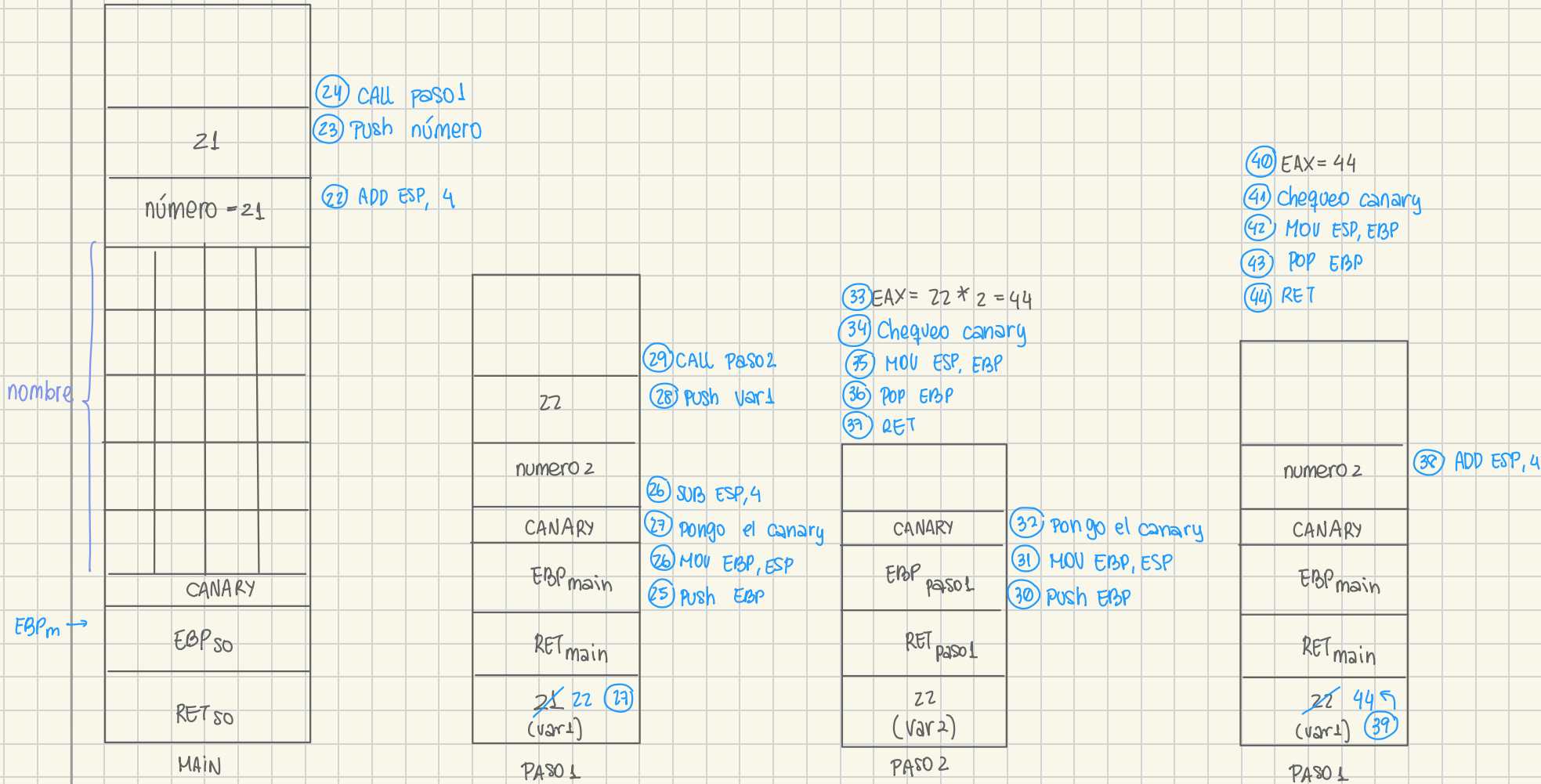
int paso1(int var1) {
    int numero2;
    var1++;
    var1=paso2(var1);
    return var1;
}

int main(void) {
    char nombre[20];
    int numero=21;
    printf("Ingrese su nombre: ");
    scanf("%s",nombre);
    numero=paso1(numero);
    printf("\nSu nombre es %s y su número es %d\n",nombre,numero);
    return 0;
}
```

- Explique con dibujos respectivos el contenido de la pila, durante toda la ejecución del programa para las tres funciones main, paso1 y paso2
- Muestre los registros del microprocesador que intervienen en los dibujos de la pila.
- Muestre los valores que retorna cada función y la forma en que lo hace.

siguiente instrucción





Dado el siguiente código en C

```

void encrypt(char *plain, char *cipher) {
    if (!(*plain)){
        *cipher = 0;
        return;
    }

    char pad = get_pad();
    *cipher = *plain + pad;
    encrypt(plain+1, cipher+1);
}

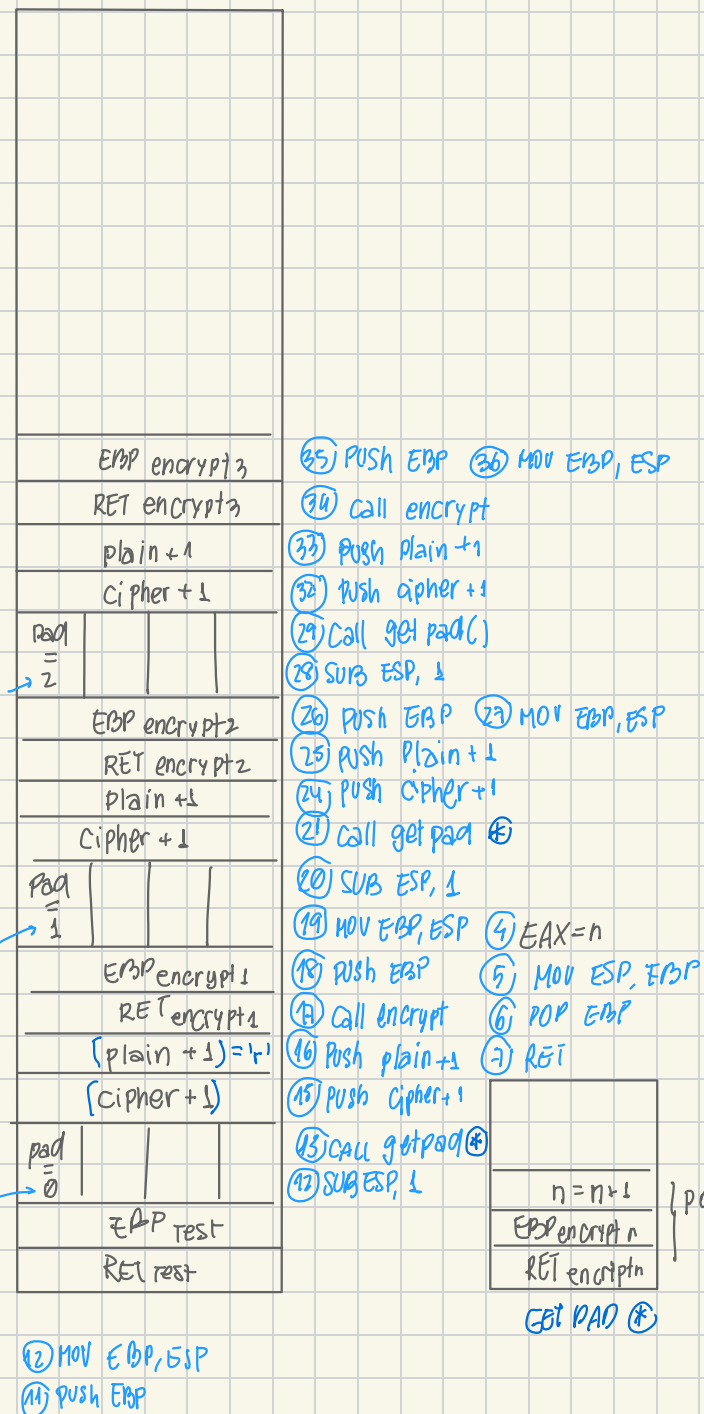
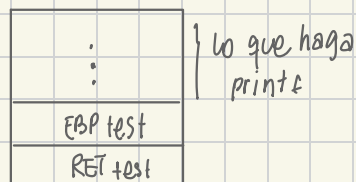
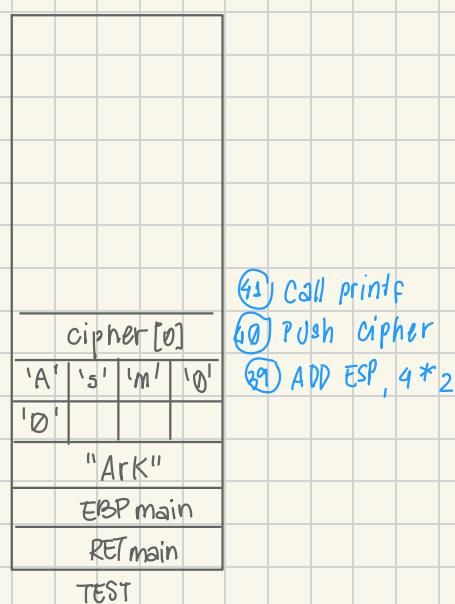
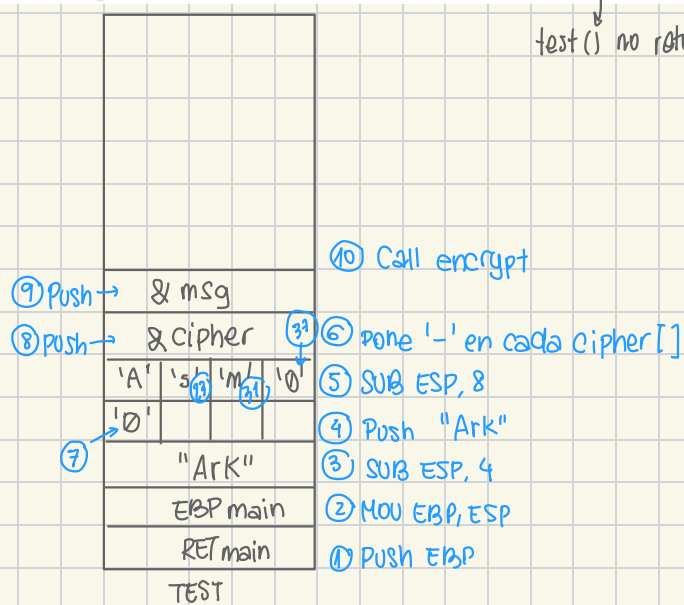
void test() {
    char *msg = "Ark";
    char cipher[] = "-----";
    cipher[4] = '0';

    encrypt(msg, cipher);

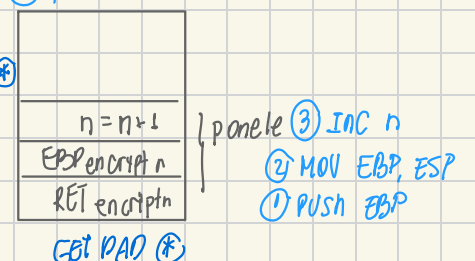
    printf(cipher);
}

```

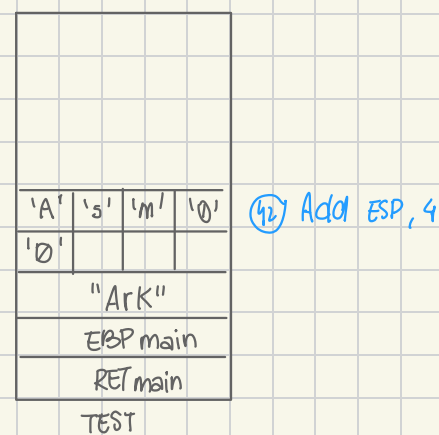
- Suponga que el dispositivo de encriptación comienza devolviendo 1 byte igual a 0 e incrementa en 1 en cada llamada.
- Muestre cómo cada operación altera el estado de la pila al invocar la función `test` y cual es la salida por `salida estándar`.
- Implemente la función `get_pad` y agregue todo lo necesario para que el código funcione correctamente a partir del diseño del Ejercicio 1.
- ¿Encuentra algún error posible en todo el código anterior?



②8) Para cada encryptn
hace los borrados de
parámetros para la siguiente
llamada, setea el ESP y
RETORNA.



SALIDA: 'ASm'



Ejercicio 3/3 (3 puntos)

Haga un diagrama de las configuraciones y realice todas las implementaciones necesarias para que el siguiente código (que está en un contexto distinto) encripte un mensaje haciendo uso de la función **encrypt** del **Ejercicio 2**.

```

; code start
mov ax, src    = plain
mov bx, dst    = cipher
int 74h
; code end

section .data
src db "Hello World"

section .bss
dst resb 36

```



section	.text
GLOBAL	.int 74h
EXTERN	encrypt

```
int fh;
```

```

push ebp
mov  ebp, esp

```

push bx

Push	ax
------	----

call encrypt

Add ESP, 4 * 2

MOV ESP, EBP

POP	EBP
-----	-----

 $i R \in T$

